华东师范大学数据学院上机实践报告

OS Lab3 页表实现机制与系统调用优化实验报告

课程名称： **操作系统**	**年级：2023**	**上机实践成绩：**
指导老师：翁楚良	**姓名**：陈子谦	
上机实践名称：页表实现优化	**学号**：10235501454	**上机实践日期**：25.5.8

一、实验目的

本实验主要探究操作系统中页表的实现机制以及用户态向内核态数据拷贝的优化方法。通过实际操作xv6操作系统，深入理解虚拟内存管理和系统调用的工作原理。操作系统内核与用户程序之间的交互需要频繁进行上下文切换，而这种切换会带来性能开销。通过建立特殊的映射机制，可以减少某些简单系统调用（如`getpid()`）的上下文切换开销，提高系统性能。同时，通过实现页表遍历和打印功能，可以直观地观察操作系统内存管理的结构，加深对虚拟内存管理机制的理解。这些知识对理解现代操作系统设计和实现高效系统服务具有重要的科学价值。

二、实验内容

本实验包含两个主要任务：

1. 实现用户态加速系统调用：为了减少系统调用的开销，特别是像`getpid()`这样的简单调用，需要为每个进程在虚拟地址`USYSCALL`处映射一个只读页面，并在该页面存储进程ID信息，使得用户程序可以直接访问而无需切换到内核态。
2. 实现页表打印功能：编写一个`vmprint()`函数，递归遍历页表结构并按照指定格式打印页表内容，以便观察内存映射关系。

在数学上，虚拟地址到物理地址的映射过程可以表示为：
$VA = \{VPN_2, VPN_1, VPN_0, offset\}$
其中RISC-V架构的页表采用三级结构：

- VPN_2：指向一级页表的索引（9位）
- VPN_1：指向二级页表的索引（9位）
- VPN_0：指向三级页表的索引（9位）
- $offset$：页内偏移（12位）

通过页表项(PTE)中的物理页号(PPN)，最终实现虚拟地址到物理地址的转换：
$PA = PTE.PPN \cdot PGSIZE + VA.offset$

三、使用环境

- 操作系统：Ubuntu 24.04 (WSL)
- 编译环境：GCC
- 实验平台：xv6-labs-2022（pgbtl分支）
- 硬件架构：RISC-V

- 开发工具：VS Code

四、实验过程和结果

4.1 实现用户态加速系统调用

首先，需要理解系统调用的执行流程。在常规情况下，用户程序调用系统调用需要通过陷入指令切换到内核态，由内核执行相应功能后再返回用户态，这个过程涉及上下文切换的开销。而对于简单的系统调用如`getpid()`，可以通过在用户空间和内核空间共享一块内存来减少这种开销。

实现步骤如下：

step1

在`proc.h`中声明一个`usyscall`结构体，用于存放共享页面。

```
// these are private to the process, so p->lock need not be held.
uint64 kstack;               // Virtual address of kernel stack
uint64 sz;                   // Size of process memory (bytes)
pagetable_t pagetable;       // User page table
struct trapframe *trapframe; // data page for trampoline.S
struct usyscall *usyscall;   // usyscall data page
struct context context;      // swtch() here to run process
struct file *ofile[NOFILE];  // Open files
struct inode *cwd;           // Current directory
char name[16];               // Process name (debugging)
};
```

step2

在(`kernel/proc.c`)中修改`allocproc`方法，仿照给`trapframe`为`p->usyscall`分配具体的物理地址，并且将进程的pid保存到这个结构体之中。

```
// Allocate a usyscall page.
// 这里的地址其实就是一个物理地址，是需要在用户页表中与逻辑地址进行映射的的地址
if((p->usyscall = (struct usyscall *)kalloc()) == 0){ // xv6中使用kalloc()分配物理内存
  freeproc(p);
  release(&p->lock);
  return 0;
}
p->usyscall->pid = p->pid;
```

step3

在`(kernel/proc.c)`中修改`proc_pagetable`方法，仿照给`trapframe`新增映射关系，这里实验有要求许用户空间只读取页面的权限位，所以使用权限`PTE_R`与`PTE_U`

```
if(mappages(pagetable, USYSCALL, PGSIZE,
            (uint64)(p->usyscall), PTE_R | PTE_U) < 0){
  uvmunmap(pagetable, USYSCALL, 1, 0);
  uvmfree(pagetable, 0);
  return 0;
}
```

step4

在(`kernel/proc.c`)中修改`freeproc`与`proc_freepagetable`，在进程释放的时候将对应内存释放掉。

`freeproc`中增加：

```
if(p->usyscall)
  kfree((void*)p->usyscall);
p->usyscall = 0;
```

`proc_freepagetable`中增加

```
void
proc_freepagetable(pagetable_t pagetable, uint64 sz)
{
  uvmunmap(pagetable, USYSCALL, 1, 0);
  uvmunmap(pagetable, TRAMPOLINE, 1, 0);
  uvmunmap(pagetable, TRAPFRAME, 1, 0);
  uvmfree(pagetable, sz);
}
```

由step2可以看出内核中其实是可以直接进行物理内存的操作，包括分配内存，和写入数据。那么这里就有个人有个疑问就是，逻辑地址的重要性到现在体现的还不是很明显，这个问题还有待回答。

还有就是step4中为什么不释放USYSCALL所在内存会导致panic？freeproc这个方法在进程的生命周期的哪个阶段会被调用，一个进程的所有内存没被全部释放掉会导致什么问题？

以上疑问有待解决。

以下是测试结果截图：

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ pgtbltest
ugetpid_test starting
ugetpid_test: OK
pgaccess_test starting
pgtbltest: pgaccess_test failed: incorrect access bits set, pid=3
$ 
```

4.2 实现页表打印功能

页表是操作系统管理虚拟内存的核心数据结构，通过实现页表打印功能，可以直观地了解内存映射关系。在RISC-V架构中，页表采用三级结构，需要递归遍历这三级结构才能完整地打印页表内容。

step1

在`vm.c`文件中实现`vmprint`函数，参照freewalk实现：

```
// pre-save point
static char *pre_save_point[] = {
  [0] = "..",
  [1] = ".. ..",
  [2] = ".. .. ..",
};

void
vmprint(pagetable_t pagetable, uint64 level)
{
  if(level == 1){
    printf("page table: %p\n", pagetable);
  }
  // there are 2^9 = 512 PTEs in a page table.
  for(int i = 0; i < 512; i++){
    pte_t pte = pagetable[i];
    if((pte & PTE_V)){
      // this PTE points to a lower-level page table.
      uint64 child = PTE2PA(pte);
      printf("%s%d: pte %p pa %p\n", pre_save_point[level-1], i, pte, child);
      if(level != 3){
        vmprint((pagetable_t)child, level+1); //避免递归过度
      }
    } else if(pte & PTE_V){
      panic("freewalk: leaf");
    }
  }
}
```

该函数采用递归方式遍历页表，当找到有效的页表项时，打印该项的信息并递归遍历下一级页表。通过不同级别的缩进（使用`pre_save_point`数组）来表示页表的层次结构。

step2

在`defs.h`中增加`vmprint`声明

```
uint64          walkaddr(pagetable_t, uint64);
int             copyout(pagetable_t, uint64, char *, uint64);
int             copyin(pagetable_t, char *, uint64, uint64);
int             copyinstr(pagetable_t, char *, uint64, uint64);
void            vmprint(pagetable_t, uint64); // new line

```

step3

在`exec.c`文件中，在`exec`函数的返回前添加对`vmprint`的调用：

```
if(p->pid == 1){
  vmprint(pagetable, 1); //只打印一级页表
}
```

此调用仅在初始进程（PID=1）执行时打印页表，避免大量输出影响系统运行。

运行测试后，可以观察到页表的打印输出，截图如下：

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
page table: 0x0000000087f6b000
..0: pte 0x0000000021fd9c01 pa 0x0000000087f67000
.. ..0: pte 0x0000000021fd9801 pa 0x0000000087f66000
.. .. ..0: pte 0x0000000021fda01f pa 0x0000000087f68000
.. .. ..1: pte 0x0000000021fd941f pa 0x0000000087f65000
.. .. ..2: pte 0x0000000021fd9007 pa 0x0000000087f64000
.. .. ..3: pte 0x0000000021fd8c17 pa 0x0000000087f63000
..255: pte 0x0000000021fda801 pa 0x0000000087f6a000
.. ..511: pte 0x0000000021fda401 pa 0x0000000087f69000
.. .. ..509: pte 0x0000000021fdcc13 pa 0x0000000087f73000
.. .. ..510: pte 0x0000000021fdd007 pa 0x0000000087f74000
.. .. ..511: pte 0x0000000020001c0b pa 0x0000000080007000
```

从输出结果可以清晰地看到三级页表的结构和映射关系。页表的每一项都显示了页表项的值(pte)和对应的物理地址(pa)，通过缩进可以清楚地区分不同级别的页表。

五、结论

本实验通过实现用户态加速系统调用和页表打印功能，深入探索了操作系统中的页表机制和系统调用优化策略。通过为每个进程映射特定的内存页，用户程序可以直接读取某些内核数据而无需执行完整的系统调用流程，从而减少了不必要的上下文切换开销。这种优化策略在实际系统中对于频繁调用的简单系统调用具有显著的性能提升。

同时，通过实现页表打印功能，可以直观地观察内存映射关系，加深对虚拟内存管理机制的理解。在RISC-V三级页表的结构中，每级页表的映射关系都清晰可见，这对于调试内存相关问题和理解内存管理原理非常有帮助。

然而，这种优化方法也存在一些局限性。首先，它只适用于简单的、不需要复杂参数和返回值的系统调用。对于需要大量参数传递或返回复杂数据结构的系统调用，这种方法的适用性较低。其次，共享内存页面的方法可能引入安全风险，如果实现不当，可能导致信息泄露或权限提升漏洞。